

DP Dot product	$C = \sum_{i=1}^n 2 = 2n = \mathcal{O}(n)$
Input: $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$ Output: $s = \mathbf{x} \cdot \mathbf{y} = \mathbf{x}^\top \mathbf{y}$ 1: $s = 0$ 2: for $i = 1, \dots, n$ do 3: $s = s + x_i y_i$ 4: end for	
Rank Theorem: $[\mathbf{A}\mathbf{x} = \mathbf{0} \text{ iff } \mathbf{x} = \mathbf{0}] \Leftrightarrow [\text{列线性无关}] \Leftrightarrow [\mathbf{A} \text{ 可逆}] \Leftrightarrow [\text{rank}(\mathbf{A}) = \text{rank}(\mathbf{A}^\top) = n]$ Abel: ≥ 5 次方 实系数多项式 $\exists$ 实根无法用有限次加減乘除和开方表示. Spectral Theorem: If $\mathbf{A}$ is symmetric, then $\exists$ orthonormal $\mathbf{Q}$ s.t. $\mathbf{A} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^\top$ (Orthogonally diagonalisable) $\mathbf{\Lambda} = \text{diag}(\lambda_1, \dots, \lambda_n)$	
LU LU factorisation	
$C(n) = \sum_{k=1}^{n-1} (n-k)(1+2(n-k+1)) = \frac{2}{3}n^3 + \frac{1}{2}n^2 - \frac{7}{6}n$	
Input: $\mathbf{A} \in \mathbb{R}^{n \times n}$ Output: $\mathbf{L}, \mathbf{U} \in \mathbb{R}^{n \times n}$ s.t. $\mathbf{A} = \mathbf{L}\mathbf{U}$ 1: $\mathbf{L} = \mathbf{I}, \mathbf{U} = \mathbf{A}$ 2: for $k = 1, \dots, n-1$ do 3: for $j = k+1, \dots, n$ do 4: $l_{jk} = u_{jk}/u_{kk}$ 5: $(u_{jk}, \dots, u_{jn}) = (u_{jk}, \dots, u_{jn}) - l_{jk}(u_{kk}, \dots, u_{kn})$ 6: end for 7: end for	
GE Gaussian elimination	$C(n) = \frac{2}{3}n^3 + \frac{5}{2}n^2 - \frac{7}{6}n = \mathcal{O}(n^3)$
Input: $\mathbf{A} \in \mathbb{R}^{n \times n}, \mathbf{b} \in \mathbb{R}^n$ Output: $\mathbf{x} \in \mathbb{R}^n$ s.t. $\mathbf{A}\mathbf{x} = \mathbf{b}$ 1: Find LU factorisation of $\mathbf{A} = \mathbf{LU}$ # LU Algorithm 2: Solve $\mathbf{Ly} = \mathbf{b}$ using forward substitution. # FS Algorithm 3: Solve $\mathbf{Ux} = \mathbf{y}$ using back substitution. # BS Algorithm	

LUPP LU factorisation with partial pivoting (not unique)	$C(n) = \frac{2}{3}n^3 + n^2 - \frac{5}{3}n$
Input: $\mathbf{A} \in \mathbb{R}^{n \times n}$ Output: $\mathbf{L}, \mathbf{U}, \mathbf{P} \in \mathbb{R}^{n \times n}$ s.t. $\mathbf{PA} = \mathbf{LU}$ with $\mathbf{L}$ 单位下三角, $\mathbf{U}$ 可逆上三角 1: $\mathbf{U} = \mathbf{A}, \mathbf{L} = \mathbf{I}, \mathbf{P} = \mathbf{I}$ 2: for $k = 1, \dots, n-1$ do 3: Choose $i \in \{k, \dots, n\}$ which maximizes $ u_{ik} $ # Max Algorithm 4: Exchange row $(u_{k1}, \dots, u_{kn})$ with $(u_{ik}, \dots, u_{in})$ # 第 k 行和第 i 行的 $k \sim n$ 列交换 5: Exchange row $(l_{k1}, \dots, l_{k,k-1})$ with $(l_{i1}, \dots, l_{i,k-1})$ # 第 k 行和第 i 行的 $1 \sim k-1$ 列交换 6: Exchange row $(p_{k1}, \dots, p_{kn})$ with $(p_{i1}, \dots, p_{in})$ # 第 k 行和第 i 行的全部列交换 7: for $j = k+1, \dots, n$ do 8: $l_{jk} = u_{jk}/u_{kk}$ 9: $(u_{jk}, \dots, u_{jn}) = (u_{jk}, \dots, u_{jn}) - l_{jk}(u_{kk}, \dots, u_{kn})$ 10: end for 11: end for	
(M)GS SQR Householder-QR	
SQR (SLE) via QR-Factorization $C(n) = 2n^3 + 4n^2 + 3n$ Input: $\mathbf{A} \in \mathbb{R}^{n \times n}, \mathbf{b} \in \mathbb{R}^n$ Output: $\mathbf{x} \in \mathbb{R}^n$ with $\mathbf{A}\mathbf{x} = \mathbf{b}$ 1: Find QR factorization of $\mathbf{A}$ # GS MGS Algorithm 2: Solve $\mathbf{Qy} = \mathbf{b}$ # MV Algorithm # ps: $\mathbf{y} = \mathbf{Q}^{-1}\mathbf{b} = \mathbf{Q}^\top \mathbf{b}$ 3: Solve $\mathbf{Rx} = \mathbf{y}$ using Algorithm BS # BS Algorithm	
Sum Notation: $\sum_{k=1}^n k = \frac{n(n+1)}{2} = \frac{1}{2}n^2 + \frac{1}{2}n$ $\sum_{k=1}^n k^2 = \frac{n(n+1)(2n+1)}{6} = \frac{1}{3}n^3 + \frac{1}{2}n^2 + \frac{1}{6}n$ $\sum_{k=1}^n k^3 = \left(\frac{n(n+1)}{2}\right)^2 = \frac{1}{4}n^4 + \frac{1}{2}n^3 + \frac{1}{4}n^2$ Rank Theorem: $\mathbf{A}_{m \times n}, m \geq n: [\mathbf{A}\mathbf{x} = \mathbf{0} \text{ iff } \mathbf{x} = \mathbf{0}] \Leftrightarrow [\text{rank}(\mathbf{A}) = n]$ Cauchy-Schwarz Inequality: For $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n, \mathbf{x} \cdot \mathbf{y} = \mathbf{x}^\top \mathbf{y} \leq \mathbf{x} _2 \mathbf{y} _2$ Determinant: $\det(\mathbf{AB}) = \det(\mathbf{A})\det(\mathbf{B})$ Orthogonal: If $\mathbf{Q}^\top \mathbf{Q} = \mathbf{I}$ Eigen: $\mathbf{A}\mathbf{v} = \lambda \mathbf{v}$ $\xi_{\mathbf{A}}(\lambda) = \det(\mathbf{A} - \lambda \mathbf{I}) = \prod_i (\lambda_i - \lambda)$ Iterative Method OP MAX	
GI JAC GAU	JAC: $C = \mathcal{O}(k_{\max}n^2)$ GAU: $C = \mathcal{O}(k_{\max}n^2)$
Input: $\mathbf{A} = \mathbf{M} + \mathbf{N} \in \mathbb{R}^{n \times n}, \mathbf{b} \in \mathbb{R}^n, \mathbf{x}_0 \in \mathbb{R}^n, \varepsilon_r > 0$ Output: $\mathbf{x}_k \in \mathbb{R}^n$ with $\mathbf{Ax}_k \approx \mathbf{b}$ 1: $k = 0$ # JAC: $C = 2n^2/\text{it}$ GAU: $C = 2n^2/\text{it}$ 2: while $\ \mathbf{Ax}_k - \mathbf{b}\ _p > \varepsilon_r$ do # e.g. $\varepsilon_r = 10^{-3} \ \mathbf{b}\ _p$ 3: $k = k + 1$ # 上面给的 C 不含这步 4: Compute $\mathbf{y}_k = \mathbf{b} - \mathbf{N}\mathbf{x}_{k-1}$ 5: Solve $\mathbf{M}\mathbf{x}_k = \mathbf{y}_k$ # $\downarrow \mathbf{A} = \mathbf{D} + \mathbf{L} + \mathbf{U}$ 6: end while # JAC: $\mathbf{M} = \mathbf{D}, \mathbf{N} = \mathbf{L} + \mathbf{U};$ GAU: $\mathbf{M} = \mathbf{D} + \mathbf{L}, \mathbf{N} = \mathbf{U}$	

LSQ-QR LSQ via QR	$C = 2mn^2 - \frac{2}{3}n^3 + \mathcal{O}(m^2 + n^2 + mn)$
Input: $\mathbf{A} \in \mathbb{R}^{m \times n}, m \geq n, \text{rank}(\mathbf{A}) = n, \mathbf{b} \in \mathbb{R}^m$ $\uparrow$ by reduced QR Output: $\mathbf{x} \in \mathbb{R}^n$ with $\ \mathbf{Ax} - \mathbf{b}\ _2$ minimised 1: Compute reduced QR factorization $\mathbf{A} = \mathbf{Q}_1 \mathbf{R}_1$ # QR Algorithm 2: Compute $\mathbf{y} = \mathbf{Q}_1^\top \mathbf{b}$ # MV Algorithm # $C = 2mn$ 3: Solve $\mathbf{R}_1 \mathbf{x} = \mathbf{y}$ # BS Algorithm # $C = n^2$	
PI Power Iteration	$C = k_{\max}(4n^2 + 5n + 1)$ (有优化过)
Input: $\mathbf{A} \in \mathbb{R}^{n \times n}$ symmetric, $\mathbf{z}^{(0)} \in \mathbb{R}^n \setminus \{\mathbf{0}\}, \varepsilon > 0$ Output: $\mathbf{z}^{(k)} \in \mathbb{R}^n, \lambda^{(k)} \in \mathbb{R}$ s.t. $\mathbf{z}^{(k)} \approx \mathbf{x}_{\max}, \lambda^{(k)} \approx \lambda_{\max}$ 1: $k = 0$ 2: $\lambda^{(0)} = r_{\mathbf{A}}(\mathbf{z}^{(0)})$ 3: while $\ \mathbf{Az}^{(k)} - \lambda^{(k)}\mathbf{z}^{(k)}\ _2 > \varepsilon$ do # or: $ \lambda^{(k)} - \lambda^{(k-1)} > \varepsilon$ 4: $k = k + 1$ # or: $\ \mathbf{z}^{(k)} - \mathbf{z}^{(k-1)}\ _2 > \varepsilon$ 5: $\mathbf{z}^{(k)} = \mathbf{Az}^{(k-1)}$ 6: $\mathbf{z}^{(k)} = \frac{\mathbf{z}^{(k)}}{\ \mathbf{z}^{(k)}\ _2}$ # To improve stability and avoid large values 7: $\lambda^{(k)} = r_{\mathbf{A}}(\mathbf{z}^{(k)})$ # In fact: $\mathbf{z}^{(k)} = \frac{\mathbf{A}^k \mathbf{z}^{(0)}}{\ \mathbf{A}^k \mathbf{z}^{(0)}\ _2}$ and $\lambda^{(k)} = r_{\mathbf{A}}(\mathbf{z}^{(k)})$ 8: end while	

MV Matrix-vector multiplication	
$C = \sum_{i=1}^m 2n = 2nm = \mathcal{O}(mn)$	
Input: $\mathbf{A} \in \mathbb{R}^{m \times n}, \mathbf{x} \in \mathbb{R}^n$ Output: $\mathbf{y} = \mathbf{Ax}$ 1: for $i = 1, \dots, m$ do 2: $y_i = \mathbf{a}_i^\top \mathbf{x}$ # DP Algorithm 3: end for	

FS Forward substitution	
$C(n) = \sum_{j=1}^n [2(j-1) + 1] = n^2 = \mathcal{O}(n^2)$	
Input: $\mathbf{L} \in \mathbb{R}^{n \times n}$ (lower triangular), $\mathbf{b} \in \mathbb{R}^n$ Output: $\mathbf{y} \in \mathbb{R}^n$ such that $\mathbf{Ly} = \mathbf{b}$ 1: for $j = 1, \dots, n$ do 2: $y_j = \frac{1}{l_{jj}} \left(b_j - \sum_{k=1}^{j-1} l_{jk} y_k \right)$ 3: end for	

GEPP GE(partial pivoting)	$C(n) = \frac{2}{3}n^3 + 3n^2 - \frac{5}{3}n = \mathcal{O}(n^3)$
Input: $\mathbf{A} \in \mathbb{R}^{n \times n}, \mathbf{b} \in \mathbb{R}^n$ # numpy.linalg.solve use it Output: $\mathbf{x} \in \mathbb{R}^n$ satisfying $\mathbf{Ax} = \mathbf{b}$ 1: Find $\mathbf{PA} = \mathbf{LU}$ using Algorithm LUPP # LUPP Algorithm 2: Solve $\mathbf{Ly} = \mathbf{Pb}$ by forward substitution # FS Algorithm 3: Solve $\mathbf{Ux} = \mathbf{y}$ by back substitution # BS Algorithm	

LUCP LU factorisation with complete pivoting (not unique)	$C(n) = n^3 + n^2 - 2n$
Input: $\mathbf{A} \in \mathbb{R}^{n \times n}$ Output: $\mathbf{L}, \mathbf{U}, \mathbf{P}, \mathbf{Q} \in \mathbb{R}^{n \times n}$ s.t. $\mathbf{PAQ} = \mathbf{LU}$ with $\mathbf{L}$ 单位下三角, $\mathbf{U}$ 可逆上三角 1: $\mathbf{U} = \mathbf{A}, \mathbf{L} = \mathbf{I}, \mathbf{P} = \mathbf{I}, \mathbf{Q} = \mathbf{I}$ 2: for $k = 1, \dots, n-1$ do 3: Choose $(i, j) \in \{(k, k), \dots, (n, n)\}$ which maximizes $ u_{ij} $ # Max Algorithm 4: Exchange row $(u_{k1}, \dots, u_{kn})$ with $(u_{ik}, \dots, u_{in})$ # 第 k 行和第 i 行的 $k \sim n$ 列交换 U 5: Exchange row $(l_{k1}, \dots, l_{k,k-1})$ with $(l_{i1}, \dots, l_{i,k-1})$ # 第 k 行和第 i 行的 $1 \sim k-1$ 列交换 L 6: Exchange row $(p_{k1}, \dots, p_{kn})$ with $(p_{i1}, \dots, p_{in})$ # 第 k 行和第 i 行的全部列交换 P 7: Exchange column $(u_{1k}, \dots, u_{nk})$ with $(u_{1j}, \dots, u_{nj})$ # 第 k 列和第 j 列的全部行交换 U 8: Exchange column $(q_{1k}, \dots, q_{nk})$ with $(q_{1j}, \dots, q_{nj})$ # 第 k 列和第 j 列的全部行交换 Q 9: for $m = k+1, \dots, n$ do 10: $l_{mk} = u_{mk}/u_{kk}$ 11: $(u_{mk}, \dots, u_{mn}) = (u_{mk}, \dots, u_{mn}) - l_{mk}(u_{kk}, \dots, u_{kn})$ 12: end for 13: end for	

(M)GS (Modified) Gram-Schmidt	$C(n) = 2n^3 + n^2 + n$
Input: $\mathbf{A} \in \mathbb{R}^{n \times n}$, with columns $\mathbf{a}_1, \dots, \mathbf{a}_n$ Output: $\mathbf{Q} \in \mathbb{R}^{n \times n}$, with orthonormal columns $\mathbf{q}_1, \dots, \mathbf{q}_n$ $\mathbf{R} \in \mathbb{R}^{n \times n}$ non-singular upper triangular matrix; s.t. $\mathbf{A} = \mathbf{QR}$ 1: $\mathbf{R} = \mathbf{0}$ 2: for $j = 1, \dots, n$ do 3: $\mathbf{q}_j = \mathbf{a}_j$ 4: for $k = 1, \dots, j-1$ do 5: $r_{kj} = \mathbf{q}_k^\top \mathbf{a}_j$ $(r_{kj} = \mathbf{q}_k^\top \mathbf{q}_j)$ 6: $\mathbf{q}_j = \mathbf{q}_j - r_{kj} \mathbf{q}_k$ 7: end for 8: $r_{jj} = \ \mathbf{q}_j\ _2 = \sqrt{\mathbf{q}_j^\top \mathbf{q}_j}$ 9: $\mathbf{q}_j = \frac{\mathbf{q}_j}{r_{jj}}$ 10: end for	
OP Outer product	$C = \sum_{i=1}^n \sum_{j=1}^n 1 = n^2 = \mathcal{O}(n^2)$
Input: $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$ Output: $\mathbf{A} = \mathbf{xy}^\top$ 1: for $i = 1, \dots, n$ do 2: for $j = 1, \dots, n$ do 3: $a_{ij} = x_i y_j$ 4: end for 5: end for	

LSQ-SVD LSQ via SVD	
Input: $\mathbf{A} \in \mathbb{R}^{m \times n}, m \geq n, \text{rank}(\mathbf{A}) = n, \mathbf{b} \in \mathbb{R}^m$ Output: $\mathbf{x} \in \mathbb{R}^n$ with $\ \mathbf{Ax} - \mathbf{b}\ _2$ minimised 1: Compute reduced SVD $\mathbf{A} = \mathbf{U}_1 \Sigma_1 \mathbf{V}^\top$ # see SVD note 2: Compute $\mathbf{z} = \mathbf{U}_1^\top \mathbf{b}$ # MV Algorithm # $C = 2mn$ 3: Solve $\Sigma_1 \mathbf{y} = \mathbf{z}$ # $C = n$ 4: Compute $\mathbf{x} = \mathbf{Vy}$ # MV Algorithm # $C = 2n^2$	

SII Shifted Inverse Iteration	$C = \mathcal{O}(n^3) + (k_{\max} - 1)\mathcal{O}(n^2)$ (有优化过)
Input: $\mathbf{A} \in \mathbb{R}^{n \times n}$ symmetric, $s \in \mathbb{R}, \mathbf{z}^{(0)} \in \mathbb{R}^n \setminus \{\mathbf{0}\}, \varepsilon > 0$ Output: $\mathbf{z}^{(k)} \in \mathbb{R}^n, \lambda^{(k)} \in \mathbb{R}$ s.t. $\mathbf{z}^{(k)} \approx \mathbf{x}_\alpha, \lambda^{(k)} \approx \lambda_\alpha$ near s 1: $k = 0$ 2: $\lambda^{(0)} = r_{\mathbf{A}}(\mathbf{z}^{(0)})$ # first $\mathcal{O}(n^3)$ cost; then $\mathcal{O}(n^2)$ cost/it 3: while $\ \mathbf{Az}^{(k)} - \lambda^{(k)}\mathbf{z}^{(k)}\ _2 > \varepsilon$ do 4: $k = k + 1$ 5: Solve $(\mathbf{A} - s\mathbf{I})\mathbf{y} = \mathbf{z}^{(k-1)}$ for $\mathbf{y}$ # via GEPP & Find LU only once 6: $\mathbf{z}^{(k)} = \frac{\mathbf{y}}{\ \mathbf{y}\ _2}$ 7: $\lambda^{(k)} = r_{\mathbf{A}}(\mathbf{z}^{(k)})$ # In fact: $\mathbf{z}^{(k)} = \frac{(\mathbf{A} - s\mathbf{I})^{-k} \mathbf{z}^{(0)}}{\ (\mathbf{A} - s\mathbf{I})^{-k} \mathbf{z}^{(0)}\ _2}, \lambda^{(k)} = r_{\mathbf{A}}(\mathbf{z}^{(k)})$ 8: end while	

MM Matrix-matrix multiplication	
$C = \sum_{i=1}^m \sum_{j=1}^p 2n = 2mnp = \mathcal{O}(mnp)$	
Input: $\mathbf{A} \in \mathbb{R}^{m \times n}, \mathbf{B} \in \mathbb{R}^{n \times p}$ Output: $\mathbf{C} = \mathbf{AB}$ 1: for $i = 1, \dots, m$ do 2: for $j = 1, \dots, p$ do 3: $c_{ij} = \mathbf{a}_i^\top \mathbf{b}_j$ # DP Algorithm 4: end for 5: end for	

BS Back substitution	
$C(n) = \sum_{j=1}^n [2(n-j) + 1] = n^2 = \mathcal{O}(n^2)$	
Input: $\mathbf{U} \in \mathbb{R}^{n \times n}$ (upper triangular), $\mathbf{y} \in \mathbb{R}^n$ Output: $\mathbf{x} \in \mathbb{R}^n$ such that $\mathbf{Ux} = \mathbf{y}$ 1: for $j = n, \dots, 1$ do 2: $x_j = \frac{1}{u_{jj}} \left(y_j - \sum_{k=j+1}^n u_{jk} x_k \right)$ 3: end for	

GECP GE(complete pivoting)	$C(n) = n^3 + 3n^2 - 2n = \mathcal{O}(n^3)$
Input: $\mathbf{A} \in \mathbb{R}^{n \times n}, \mathbf{b} \in \mathbb{R}^n$ Output: $\mathbf{x} \in \mathbb{R}^n$ satisfying $\mathbf{Ax} = \mathbf{b}$ 1: Find $\mathbf{PAQ} = \mathbf{LU}$ using Algorithm LUCP # LUCP Algorithm 2: Solve $\mathbf{Ly} = \mathbf{Pb}$ by forward substitution # FS Algorithm 3: Solve $\mathbf{Uz} = \mathbf{y}$ by back substitution # BS Algorithm 4: Compute $\mathbf{x} = \mathbf{Qz}$	

QR Householder QR factorization	$C(n) = \frac{8}{3}n^3 + \mathcal{O}(n^2)$
Input: $\mathbf{A} \in \mathbb{R}^{n \times n}$ Output: $\mathbf{Q} \in \mathbb{R}^{n \times n}$ orthogonal $\mathbf{R} \in \mathbb{R}^{n \times n}$ non-singular upper triangular matrix; s.t. $\mathbf{A} = \mathbf{QR}$ 1: $\mathbf{Q} = \mathbf{I}, \mathbf{R} = \mathbf{A}$ # 如果 $\mathbf{A} \in \mathbb{R}^{m \times n}, m \geq n$, loop 到 n , 且 3.6.7 的 n 改为 m 2: for $k = 1, \dots, n-1$ do 3: $\mathbf{u} = (r_{kk}, \dots, r_{nk}) \in \mathbb{R}^{n-k+1}$ 4: $\mathbf{v} = \mathbf{u} + \text{sign}(u_1) \ \mathbf{u}\ _2 \mathbf{e}_1$ # line.4-7 得到的 $\mathbf{Q}_k$: 5: $\mathbf{v} = \frac{\mathbf{v}}{\ \mathbf{v}\ _2}$ # 使得 $\mathbf{Q}_k \mathbf{u} = -\text{sign}(u_1) \ \mathbf{u}\ _2 \mathbf{e}_1$ 6: $\mathbf{H}_k = \mathbf{I} - 2\mathbf{w}\mathbf{w}^\top \in \mathbb{R}^{(n-k+1) \times (n-k+1)}$ # line.6-8 一起算, 见下 7: $\mathbf{Q}_k = \mathbf{I}_{k-1} \oplus \mathbf{H}_k \in \mathbb{R}^{n \times n}$ # l. line.6-8 $C = 4(n-k+1)^2 + (n-k+1)$ 8: $\mathbf{R} = \mathbf{Q}_k \mathbf{R}$ # 用矩阵分块, 只有右下角 $\mathbf{H}_k \mathbf{R}_{k:n, k:n}$ 要算 $\Rightarrow$ 用 $\mathbf{H}_k$ 定义 Alg. MV+OP 9: $\mathbf{Q} = \mathbf{QQ}_k$ # If using in Alg. SQR , then $\mathbf{y}' : \mathbf{y} = \mathbf{Q}_k \mathbf{y}$; $C = \frac{4}{3}n^3 + \mathcal{O}(n^2)$ (SQR/QR) 10: end for # $\mathbf{A} \in \mathbb{R}^{m \times n}$ reduced 的 $C = 2n^2m - \frac{2}{3}n^3 + \mathcal{O}(m^2 + n^2 + mn)$	
Max Finding index of maximum	$C(m) = m - 1$
Input: $\mathbf{z} \in \mathbb{R}^m$ Output: index i with $ z_i = \max_{j=1, \dots, m} z_j $ 1: $i = 1, \max = z_1 $ 2: for $j = 2, \dots, m$ do 3: if $ z_j > \max$ then 4: $\max = z_j $ 5: end if 6: end for	

QR-Eig QR Algorithm for Eigenvalues	$C =$ 和 OI Alg. 类似
Input: $\mathbf{A} \in \mathbb{R}^{n \times n}$ symmetric, $\varepsilon > 0$ Output: $\Lambda^{(k)} \in \mathbb{R}^{n \times n}$ with diagonal entries $\approx$ eigenvalues of $\mathbf{A}$. 1: $k = 0, \Lambda^{(0)} = \mathbf{A}$ 2: while $\ \Lambda^{(k)} - \Lambda^{(k-1)}\ _1 > \varepsilon$ or $k = \mathbf{0}$ do 3: $k = k + 1$ # By def $\mathbf{Q}^{(k)} = (\mathbf{Z}^{(k-1)})^\top \mathbf{Z}^{(k)}, \mathbf{Z}^{(k)}$ from OI Alg. 4: Compute QR factorization $\Lambda^{(k-1)} = \mathbf{Q}^{(k)} \mathbf{R}^{(k)}$ 5: $\Lambda^{(k)} = \mathbf{R}^{(k)} \mathbf{Q}^{(k)}$ 6: end while # Can solving $(\mathbf{A} - \lambda \mathbf{I})\mathbf{x} = \mathbf{0}$ to find eigenvector $\mathbf{x}$	
OI Orthogonal Iteration	$C =$ 取决于 QR 分解方法 $\mathbf{A}$ 计算方法
Input: $\mathbf{A} \in \mathbb{R}^{n \times n}$ symmetric, $\mathbf{Z}^{(0)} \in \mathbb{R}^{n \times n}$ orthogonal, $\varepsilon > 0$ Output: $\mathbf{Z}^{(k)} \in \mathbb{R}^{n \times n}, \Lambda^{(k)} \in \mathbb{R}^{n \times n}$ s.t. $\mathbf{Z}^{(k)} \approx [\mathbf{x}_1, \dots, \mathbf{x}_n], \Lambda^{(k)} \approx \text{diag}(\lambda_1, \dots, \lambda_n)$ 1: $k = 0$ 2: $\Lambda^{(0)} = (\mathbf{Z}^{(0)})^\top \mathbf{AZ}^{(0)}$ 3: while $\ \mathbf{AZ}^{(k)} - \Lambda^{(k)}\mathbf{Z}^{(k)}\ _1 > \varepsilon$ do # or: $\ \Lambda^{(k)} - \Lambda^{(k-1)}\ _1 > \varepsilon$ 4: $k = k + 1$ 5: Compute $\mathbf{W}^{(k)} = \mathbf{AZ}^{(k-1)}$ 6: Compute QR factorization $\mathbf{W}^{(k)} = \mathbf{Z}^{(k)} \mathbf{R}^{(k)}$ 7: $\Lambda^{(k)} = (\mathbf{Z}^{(k)})^\top \mathbf{AZ}^{(k)} = \Lambda_{\frac{1}{2}i}^{(k)} = r_{\mathbf{A}}(z_i^{(k)})$ 8: end while # $\mathbf{Z}^{(k)}$ 第一列为 P1 结果	